Marmara Üniversitesi

Bilgisayar Mühendisliği Bölümü

BLM2008 – Mikroişlemciler
PROJE RAPORU

PROJE ADI: ASTEROID ESCAPE (8086 ASSEMBLY)

Hazırlayan: İdil ŞEN- 170424067
Ders Sorumlusu:Dr. Öğr. Üyesi Serkan AYDIN

İÇİNDEKİLER

1. **Giriş ve Amaç 2**
2. **Sistem Mimarisi ve Teknik Tasarım 2**
   - **2.1. Grafik Alt Yapısı ve Video Bellek Yönetimi 2**
   - **2.2. Veri Yapıları ve Değişken Tanımları 2**
3. **Algoritmik Süreçler 3**
   - **3.1. Pseudo-Random Üretim Mekanizması 3**
   - **3.2. Çarpışma Denetimi (Collision Detection) 4**
   - **3.3. Klavye Kesme Yönetimi (Non-blocking I/O) 5**
4. **Teknik Zorluklar ve Çözümler 6**
5. **Sonuç 7**
6. **Kaynakça ……8**

1. GİRİŞ VE AMAÇ

Bu çalışma, Intel 8086 işlemci mimarisi üzerinde düşük seviyeli programlama teknikleri kullanılarak interaktif bir oyun motoru tasarlanmasını kapsamaktadır. Proje süresince, işlemci kaydedicileri (registers), BIOS ve DOS kesmeleri (interrupts), bellek adresleme ve donanım kontrolü gibi bilgisayar mühendisliğinin temel yapı taşları uygulama düzeyinde incelenmiştir. "Asteroid Escape", gerçek zamanlı veri işleme ve görselleştirme süreçlerinin kısıtlı donanım kaynakları altında nasıl optimize edilebileceğini göstermeyi amaçlamaktadır.

2. SİSTEM MİMARİSİ VE TEKNİK TASARIM

2.1. Grafik Alt Yapısı ve Video Bellek Yönetimi

Uygulama, BIOS'un `INT 10h` kesmesi üzerinden yapılandırılmıştır.

- Video Modu: Proje başlangıcında `AX=0003h` parametresi ile 80x25 boyutlarında, 16 renk destekli standart metin modu aktif edilmiştir.
- Kürsör Manipülasyonu: Kullanıcı deneyimini bozmamak adına sistem kürsörü, tarama satırları (`CH=26h`, `CL=07h`) değiştirilerek kullanıcıya görünmez hale getirilmiştir.
- Titreşim (Flicker) Analizi ve Çözümü: Assembly tabanlı oyunlarda en büyük problem olan ekran titreşimi, `AX=0600h` fonksiyonu ile tüm ekranın yukarı kaydırılması (scrolling) ve her karede yalnızca değişen karakterlerin güncellenmesi stratejisiyle minimize edilmiştir.

2.2. Veri Yapıları ve Değişken Tanımları

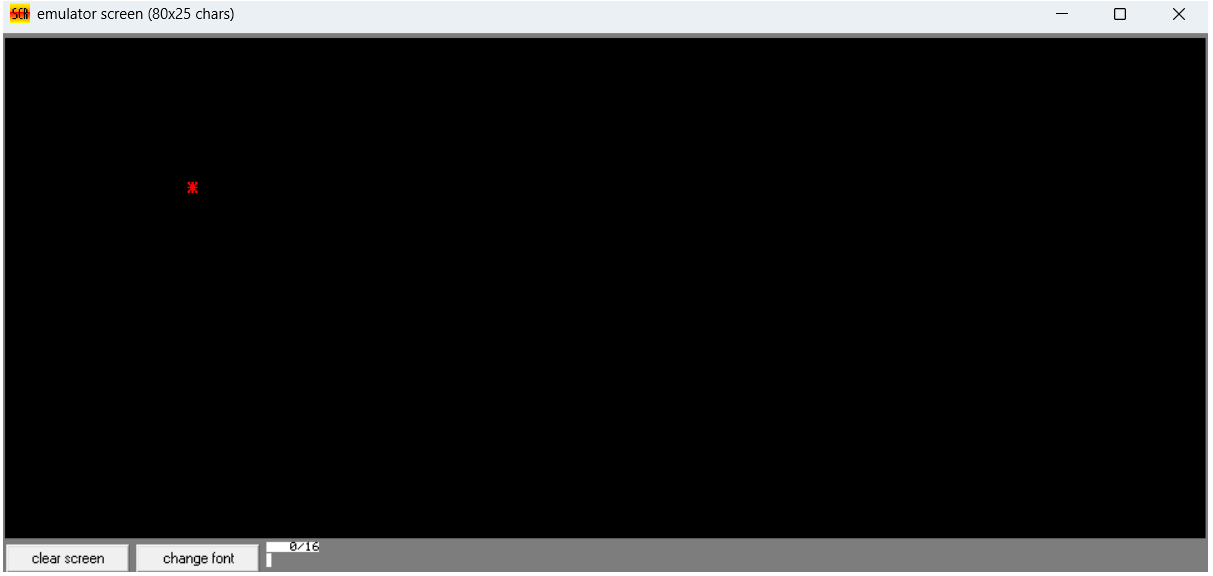
Programda kullanılan temel değişkenler 8-bit (byte) uzunluğundadır:

- `p_x` ve `p_y`: Oyuncu gemisinin ekran üzerindeki X-Y koordinatlarını tutar.
- `ast_x`, `ast_y` ve `ast_v`: Düşen objelerin yatay konumunu, dikey konumunu ve hızını kontrol eder.
- `has_shield`: Mantıksal bir flag olup, kalkan bonusunun aktiflik durumunu simgeler.

3. ALGORİTMİK SÜREÇLER

3.1. Pseudo-Random Üretim Mekanizması

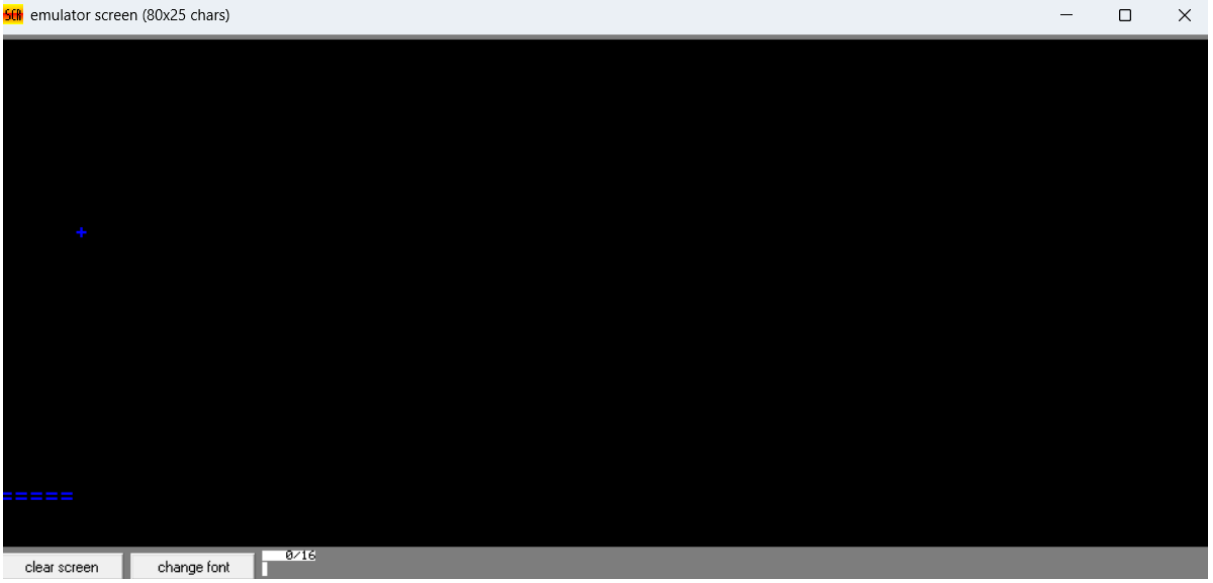
Intel 8086 işlemcisinde donanımsal bir rastgele sayı üreticisi bulunmadığından, sistem saati (`INT 1Ah`) bir "seed" (tohum) olarak kullanılmıştır. `CX:DX` kaydedicilerinden dönen milisaniye verileri üzerinde `AND` maskeleme işlemi uygulanarak, asteroidin X koordinatı ve objenin türü (Normal, Hızlı veya Kalkan) belirlenmektedir.



Şekil 1: Oyunun genel oynanış arayüzü ve farklı asteroid tiplerinin (Normal, Hızlı, Kalkan) görünümü.

3.2. Çarpışma Denetimi (Collision Detection)

Çarpışma denetimi, her oyun döngüsünde (frame) gerçekleştirilen bir karşılaştırma dizisidir. Eğer `ast_y` değeri `p_y` değerine eşitse, `ast_x` değerinin oyuncunun 5 birimlik genişlik aralığında olup olmadığı denetlenir. Kalkan aktifse çarpışma asteroidi yok eder ancak oyun sonlanmaz.

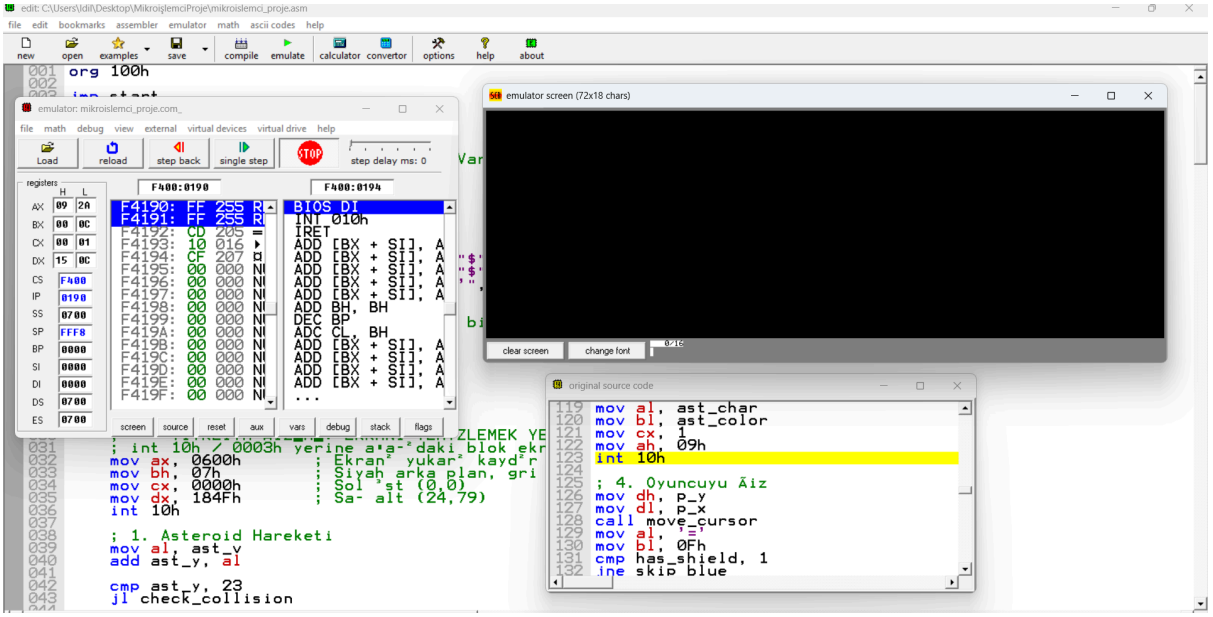


Şekil 2: Kalkan bonusunun aktifleşmesi sonucu oyuncu renginin değişimi

Şekil 3: Çarpışma denetimi testi.

3.3. Klavye Kesme Yönetimi (Non-blocking I/O)

Oyun akışının kesilmemesi için `INT 16h / AH=01h` fonksiyonu ile klavye tamponu kontrol edilir. Eğer bir girdi varsa `AH=00h` ile okunur, aksi halde oyun döngüsü `INT 15h` geciktirme fonksiyonuna devam eder.



Şekil 4: Asteroid Escape Oyun Arayüzü ve Dinamik Nesne Yönetimi

Açıklama: "Yukarıdaki görselde, oyunun 80x25 metin modunda (Text Mode) çalışma anı görülmektedir. Görselde;

- **Oyuncu Karakteri:** Alt satırda bulunan ve `p_x` değişkeni ile yatay eksende kontrol edilen raket yapısı,
- **Düşen Objeler:** `ast_x` ve `ast_y` koordinatları ile takip edilen, rastgele (pseudo-random) olarak üretilmiş asteroidler ve bonus ögeler,
- **Görsel Optimizasyon:** `INT 10h / AX=0600h` kesmesi sayesinde ekranın tamamını temizlemeden gerçekleştirilen akıcı grafik güncellemeleri gözlemlenmektedir. Görüntüde karakterlerin renkleri, BIOS attribute byte manipülasyonu ile her bir karaktere özel olarak atanmıştır."

4. TEKNİK ZORLUKLAR VE ÇÖZÜMLER

- Hız Sabitleme: İşlemci hızı döngüyü çok hızlı çalıştırdığı için `INT 15h` kesmesiyle mikro saniye düzeyinde gecikmeler eklenerek oyun oynanabilir seviyeye çekilmiştir.
- Bellek Erişimi: Segment registerlarının (`DS`, `ES`) yönetimi sağlanarak stringlerin ve değişkenlerin adres çakışmaları engellenmiştir.

Şekil 5: Asteroid çarpışması sonrası tetiklenen 'Game Over' ekranı ve kullanıcı etkileşim arayüzü.

5. SONUÇ

"Asteroid Escape" projesi, mikroişlemci düzeyinde donanım-yazılım etkileşimini anlamak adına kritik bir deneyim sağlamıştır. Assembly dilinin sunduğu doğrudan bellek erişimi ve kesme kontrolü olanakları, modern yazılımların temelindeki alt yapı mantığını derinlemesine kavramamı sağlamıştır.

6.KAYNAKÇA

1. Hall, D. V. (1992). *Microprocessors and Interfacing: Programming and Hardware.* McGraw-Hill. (8086 mimarisi ve donanım kesmeleri üzerine temel referans kaynağı).
2. Abel, P. (2001). *IBM PC Assembly Language and Programming.* Prentice Hall. (BIOS INT 10h ve INT 16h kesmelerinin detaylı kullanımı için yararlanılmıştır).
3. Irvine, K. R. (2014). *Assembly Language for x86 Processors.* Pearson. (Oyun döngüsü mantığı ve düşük seviyeli veri yönetimi stratejileri üzerine teorik altyapı).
4. Intel Corporation (1979). *8086 Family User's Manual.* (İşlemci kaydedicileri ve instruction set (komut seti) mimarisi için resmi teknik dokümantasyon).
5. Duntemann, J. (2000). *Assembly Language Step-by-Step.* Wiley. (Gecikme (delay) algoritmaları ve bellek adresleme modları üzerine pratik çözümler).
6. Microsoft Corporation. *MS-DOS Programmer's Reference.* (INT 21h servisleri ve program sonlandırma prosedürleri).